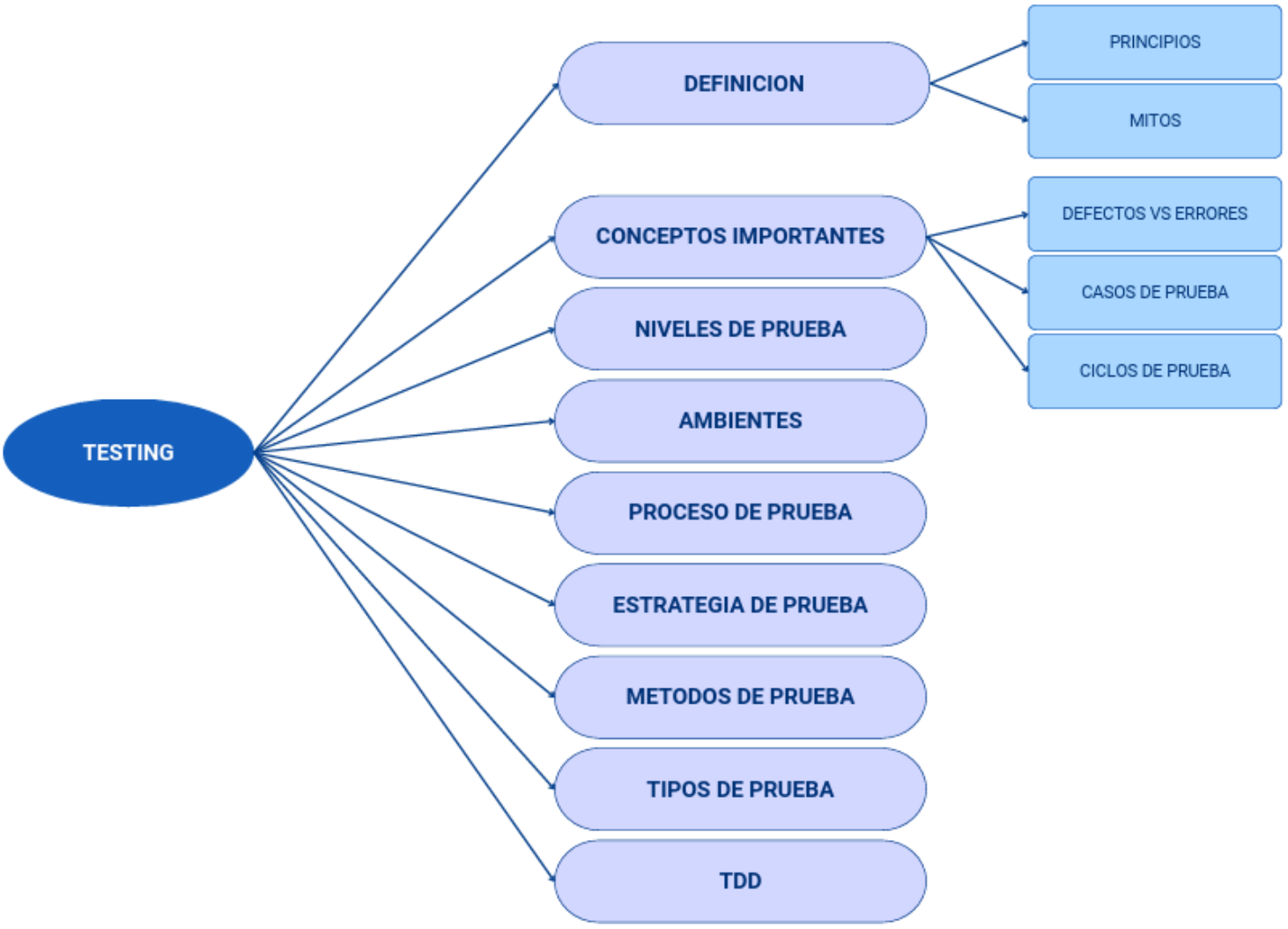
TESTING DE SOFTWARE



Como sabemos, el PUD sigue el siguiente workflow:

1. requerimientos
2. análisis
3. diseño
4. implementación (TDD)
5. **prueba**
6. despliegue

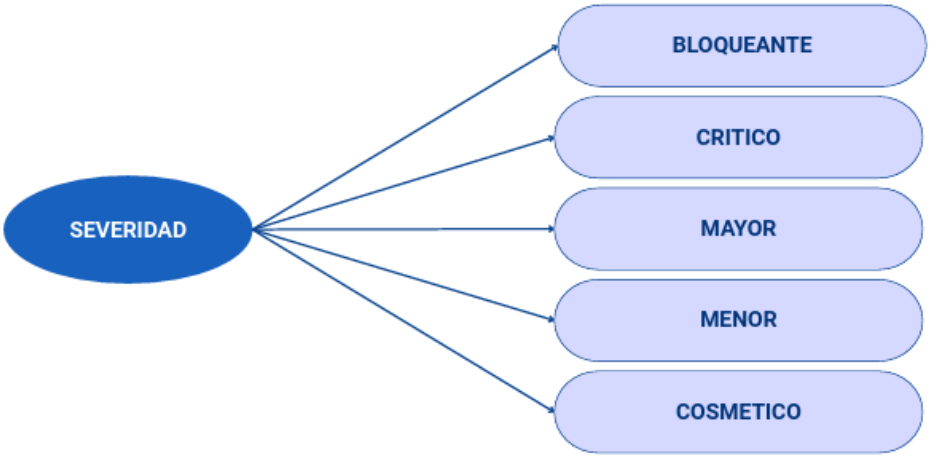
Ahora, en específico, nos ubicamos en el flujo 5 de prueba.
La prueba de software o testing está en el contexto del aseguramiento de la calidad, si bien no nos referimos al control de calidad de proceso ni de producto, está bajo el paraguas de lo que es la gestión de la configuración.
Cuando hablamos de prueba de software no nos referimos ni a Control de Calidad de Proceso, ni a Control de Calidad de Producto, es un tema en sí mismo.

¿Qué es el testing?
Se suele confundir con la calidad de software, pero tenemos que entender que el testing NO ASEGURA CALIDAD. El testing llega tarde en el ciclo.
La calidad debería asegurarse desde el principio, aparte que, quien asegura la calidad es el QA (aseguramiento de calidad) que se hace durante todo el proceso.

Entonces, el **testing** puede definirse como: "*proceso destructivo que trata de encontrar defectos en el software, su objetivo no es asegurar que el software funcione, sino encontrar defectos, para lo que asumimos que en el software HAY defectos*".
Para que un testing sea exitoso, debemos de encontrar errores. En caso contrario, si no hay un testing exitoso, es por que el desarrollo fue exitoso. Aun así, no debemos de pensar que no hay defectos ni verificar que no los hay, sino empezar con la actitud negativa que el software es incorrecto.
La persona que lleva a cabo los test no debería de ser quien lo desarrolla.

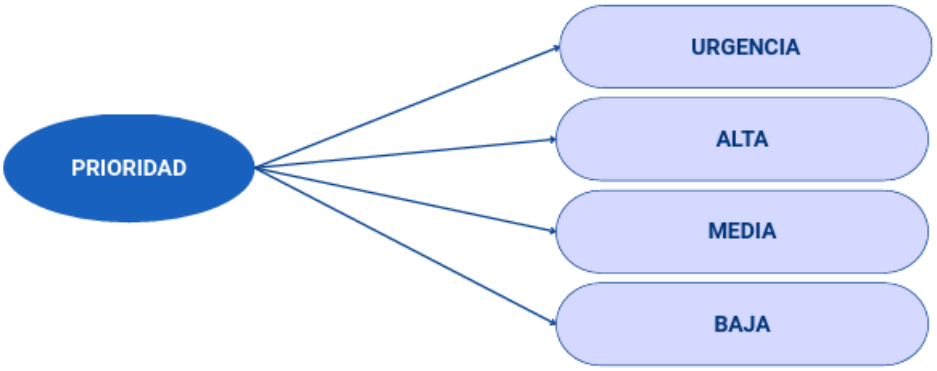
¿Cuál es la diferencia entre un error y un defecto?
- **Error**: es una equivocación que se introduce y se detecta en la misma etapa (ejemplo: error de requerimientos encontrado en el mismo análisis)
- **Defecto**: es un error que se traslada a una etapa posterior donde se introdujo. Estos defectos son los que se encuentran en la etapa de prueba.
- **Falla**: es un mal funcionamiento del sistema. Esto es la consecuencia de un defecto que no se detecta o no se arregla → no todo defecto provoca una falla, puede estar en código que nunca se ejecuta o en una condición que nadie dispara

En el testing encontramos defectos: cosas incorrectas que se originaron en etapas anteriores (requerimientos, diseño o implementación) y que llegaron hasta la etapa de prueba. Como suele haber muchos, se clasifican según **severidad** y **prioridad** (son independientes de esta clasificación).
- **Severidad (¿qué tan grave es el daño que causa el defecto en el sistema?)**: Tiene que ver con la gravedad del defecto que encontré en su impacto técnico.



  - **Bloqueante**: El sistema no anda, o no se puede seguir testeando → la app se cierra al abrirla
  - **Crítico**: Una funcionalidad no se puede cumplir, sin alternativa → no se puede finalizar una compra

    - **Mayor**: La funcionalidad se cumple, pero por un camino alternativo → el botón de “pagar” falla, pero se puede pagar desde el carrito
    - **Menor**: Cumple la funcionalidad, pero no de manera correcta → el total se muestra con un redondeo incorrecto
    - **Cosmético**: Detalle visual o de forma, sin afectar el funcionamiento → texto desaliñado, falta de ortografía
- **Prioridad (¿qué tan rápido hay que corregirlo?)**: Urgencia que tenemos para resolver este defecto, que suele definirla el negocio por que depende de cosas que no suelen ser técnicas



    - **Urgencia**: hay que corregirlo ya antes de seguir
    - **Alta**: se corrigen en el próximo ciclo
    - **Media**: se corrige cuando se pueda, sin apuro
    - **Baja**: se corrige si sobra tiempo (o nunca)

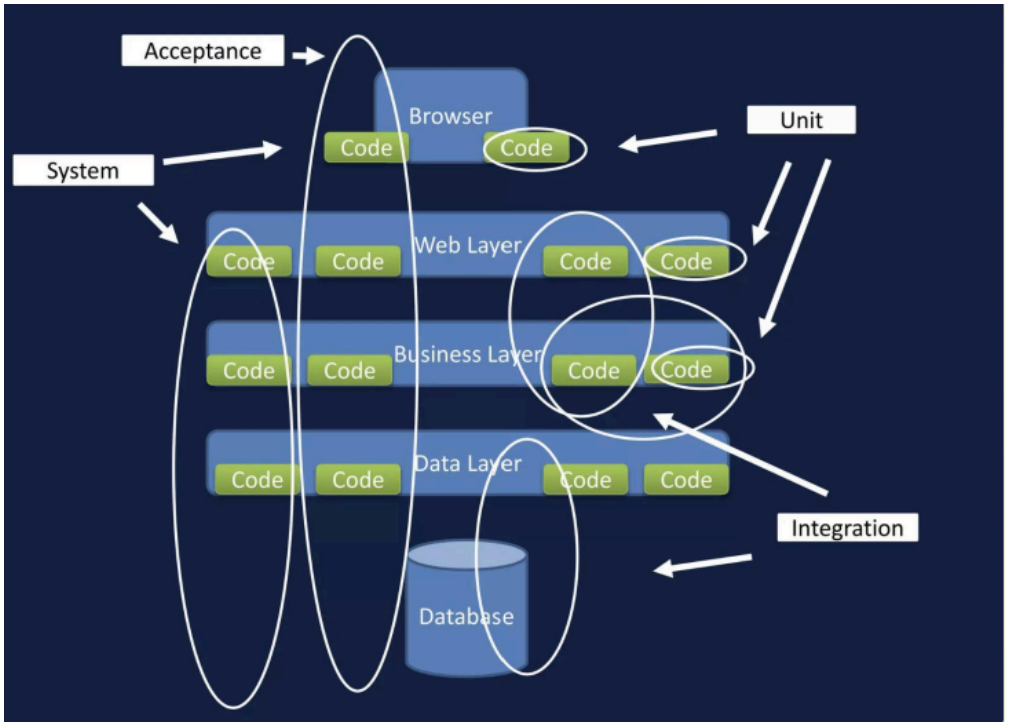
NIVELES DE PRUEBA

A la hora de realizar las pruebas, vamos a seguir un **esquema de niveles de prueba** que responde a qué tan grande es la parte del sistema que estoy probando. Se va de lo más chiquito a lo más grande, y cada nivel busca defectos distintos.

1. **UNITARIA** (desarrollador): Se prueba un componente individual, algo acotado que tiene que ver con el desarrollo que estoy realizando (una función, una clase, un método). Son respecto a aspectos puntuales y aislados del proyecto que se está realizando. Al probar componentes individuales y de forma independiente, normalmente se ejecutan por el mismo desarrollador. En estas pruebas se encuentran errores más que defectos. Se encuentra en el límite.
   Es muy fácil de automatizar
    - es lo mas chiquito → un componente por separado
    - se hace durante la implementación (TDD)
    - detecta errores de lógica interna de cada componente
2. **INTEGRACIÓN** (desarrolladores / testers): implica integrar los componentes ya probados en las pruebas unitarias (las interfaces). Se integrará para ver su funcionamiento conjunto. Normalmente requieren de un tester. Como estamos buscando verificar que las partes aisladas funcionan bien en conjunto, se hace una integración de manera incremental para lograr una identificación más correcta de los errores.
    - es la prueba de interfaces → que los componentes funcionen bien al comunicarse

- detecta defectos como datos que se pasan mal entre módulos, formatos incompatibles o llamadas mal hechas

3. **SISTEMA** (equipo de testing): Son más amplias, no solo pruebo la integración entre dos componentes, sino el sistema en toda su escala contra los requerimientos. Busca asegurarse que el sistema en su totalidad funcione de manera satisfactoria. Empiezan a impactar otras cuestiones que tienen que ver no solo con requerimientos funcionales, sino también con requerimientos no funcionales. El ambiente de prueba tiene que ser lo más parecido al entorno de producción.
   - se prueba el sistema completo, en un ambiente lo mas parecido al real
   - se verifica contra los requerimientos, tanto F como NF
   - se hace por versiones del sistema y en la etapa de prueba
   - es de caja negra
4. **ACEPTACIÓN** (usuario final / cliente): La mayoría de las veces son ejecutadas por el usuario final o el cliente. El foco no es encontrar defectos, sino que la meta es distinta, es establecer confianza en el sistema. Independientemente de que el usuario pueda encontrar defectos, no es el objetivo principal. En estas pruebas, el ambiente también tiene que ser lo más parecido posible al entorno de producción.
   - lo hace el usuario final para ver si acepta el sistema
   - no mira tanto defectos técnicos, sino mira si le drive para lo que necesita
   - se hace antes de pasar a producción → en pre-producción



AMBIENTES PARA CONSTRUIR EL SOFTWARE

Un **ambiente** es el espacio (hardware, software, configuración y datos) donde el sistema se construye, se prueba o se usa. Es en especial, por que la gente de testing necesita un ambiente limpio y controlado para que los resultados de las pruebas sean confiables.
Los ambientes son:

1. **Desarrollo (desarrolladores)**: El ambiente de desarrollo es donde los desarrolladores crean, prueban y depuran el software. Es un entorno local o en red utilizado por los desarrolladores para escribir y probar el código antes de integrarlo con el resto del sistema. Los programadores pueden utilizar herramientas de desarrollo, depuración y pruebas para garantizar que el software cumpla con los requisitos establecidos. Este ambiente suele ser flexible y permite a los desarrolladores experimentar y probar nuevas ideas sin afectar los entornos de producción. Las pruebas unitarias se llevan a cabo en el ambiente de desarrollo
   Espacio para generar el software (se programa y se hacen **pruebas unitarias**) → inestable por naturaleza, porque el código cambia todo el tiempo. Por eso no es un buen lugar para testear formalmente
2. **Prueba (testers)**: El ambiente de pruebas es donde se llevan a cabo pruebas exhaustivas del software desarrollado. Aquí se realizan pruebas de integración, pruebas funcionales, pruebas de rendimiento y otras pruebas relevantes para verificar que el software cumpla con los requisitos establecidos y funcione correctamente. El ambiente de prueba suele ser una réplica o tener características parecidas al entorno de producción, pero sin afectar a los usuarios finales y sin TODAS las características (ya que es caro tener ambientes iguales al de producción). Se utilizan conjuntos de datos y configuraciones similares a las del entorno de producción para garantizar una prueba más precisa del software. Las pruebas de integración se realizan en el ambiente de pruebas
   Espacio que se arma con las herramientas disponibles y se monta sobre un hardware donde el sistema pueda usarse → tiene que ser limpio, y es donde se hacen las **pruebas de integración y de sistema**
3. **Pre-Producción (testers - usuarios finales)**: El ambiente de preproducción, también conocido como entorno de puesta en escena o entorno de calidad, es donde se realiza una prueba final del software antes de su lanzamiento en producción. Aquí se simulan las condiciones del entorno de producción y se realizan pruebas de último minuto, como pruebas de estrés, pruebas de carga y pruebas de seguridad. El objetivo es validar y verificar que el software esté listo para ser implementado en el entorno de producción sin problemas significativos. Las pruebas de sistema se llevan a cabo en el ambiente de preproducción.
   Espacio donde se hacen las pruebas de aceptación, donde las versiones deben probarse aca antes de pasar a producción.
4. **Producción (usuario final)**: El ambiente de producción es donde el software está en funcionamiento y es accesible a los usuarios finales. Es el entorno real en el que el software se utiliza para realizar las tareas y funciones para las que fue diseñado. Este ambiente suele ser altamente controlado y está configurado para ser escalable, seguro y confiable. Se implementan medidas de respaldo y recuperación ante desastres para garantizar la disponibilidad continua del software. Es el ambiente en el que el software está funcionando. Las pruebas de aceptación se llevan a cabo en el ambiente de preproducción o en un entorno similar al de producción (en el ambiente de prueba).
   Es el ambiente real, con usuarios y datos reales.

Hoy hay **mucho costo de infraestructura**, esto implica que para tener todos estos ambientes se necesitan servidores, licencias, mantenimiento y gente que los administre, y la preproducción, que tiene que parecerse a producción, es la más cara de replicar.

CASO DE PRUEBA

Un **caso de prueba** (CP) es un conjunto de instrucciones que describe la prueba de una funcionalidad del sistema. Su buena definición nos ayuda a reproducir el defecto y reportarlo → un CP además de especificar cuál es la acción a ejecutar, debe especificar con qué datos debo realizarlo (si el caso de prueba está bien definido ayuda a identificar defectos y controlarlos, solucionarlos).

El caso de prueba tiene que ver con ejecutar una serie de pasos o acciones en una determinada funcionalidad determinando completamente con qué valores voy a hacer la ejecución para ver si el software me va a dar los resultados esperados o no.

La clave del testing es el diseño de los casos de prueba. Sin embargo, para su diseño, deben estar definidos los requerimientos (ya que de acá se suelen sacar). Si no hay una buena documentación, se tiene que poner a ver el código y aprender a crear los casos de prueba.

¿Cuáles son las partes del CP?

- **objetivo**: que quiero probar o que hecho quiero provocar en el sistema
- **Datos de entrada (condiciones de inicio)**: lo que necesito para ejecutar la prueba (datos, estado inicial del sistema, usuario de cierto rol, etc)
- **Comportamiento esperado (salida esperada)**: lo que el sistema debería hacer o devolver según los requerimientos

Como no podemos hacer un testing exhaustivo, se utilizan **métodos o estrategias de prueba** para cubrir la mayor de situaciones con la menor cantidad de pruebas. Estos métodos nos ayudan a sintetizar la elección de CP (caja negra, valores límites, entre otros).

¿De donde derivan los casos de prueba?

Los casos de prueba pueden derivar de distintos lugares, cuanta más documentación tengamos, más fácil será especificar casos de prueba.

Cuanta menos especificación tengamos, vamos a tener que describir con mayor detalle los casos de prueba.

La ideal es que vengan de los requerimientos, así podemos probar lo que el cliente pidió y no contra lo que el programador entendió.

Entre ellos:

- desde documentos del cliente
- desde información de relevamiento
- desde requerimientos
- desde especificaciones de programación
- desde el código → es en el caso que al equipo de testing no le llegue la documentación, porque deben de ponerse a ver el código y deducir el comportamiento para planear los casos. Es peor, porque probás contra lo que el sistema hace, no contra lo que debería hacer (podrías validar un defecto como si fuera correcto).

Si los requerimientos están definidos de una manera muy general y poco especificados/detallados, deberemos especificarlos ya que, si es el caso contrario, la derivación de los requerimientos hacia los casos de prueba es directa. Muchas veces se pueden utilizar los casos de prueba como especificación de requerimientos.

Lo más importante con respecto a la generación de casos de prueba es saber que ninguna técnica es completa, las distintas técnicas atacan distintos problemas y por ende es esencial

combinarlas para complementarlas ventajas de cada una y tener una especificación más detallada. Los casos de prueba dependen del código a testear y se debe tener en cuenta la conjetura de defectos. Debemos ser sistemáticos y documentar las suposiciones sobre el comportamiento o el modelo de fallas.

CONDICIONES DE PRUEBA

Tienen que ver con el contexto del sistema que tengo que tener en cuenta para poder hacer los casos de prueba. Son la reacción esperada de un sistema frente a un estímulo particular, y estando él mismo constituido por las distintas entradas.
Es algo que se enuncia ANTES de definir los casos de prueba, definir las condiciones y en función de eso defino los casos. Una condición de prueba debe ser probada por al menos un caso de prueba.

ESTRATEGIAS O MÉTODOS DE PRUEBA

El objetivo es abarcar la mayor cantidad de pruebas con el menor esfuerzo.
Las siguientes dos estrategias, que son complementarias, buscan justamente eso, con presupuesto y tiempo limitado intentan abarcar en las pruebas la mayor cantidad de software posible.

Caja Negra

No se mira el código ni como funciona el sistema por dentro, sino que se lo trata como una caja cerrada. Le vamos a dar entradas y observamos las salidas, comprandolas con lo que dice los requerimientos.

- Los **casos se derivan de los requerimientos** / especificaciones, no del código.
- Es siempre **dinámica**: hay que ejecutar el sistema.
- Se puede aplicar en **todos los niveles** (unitaria, integración, sistema y aceptación), aunque es la más usada en **sistema y aceptación**.
- **Limitación**: como no ves el código, no sabés qué partes quedaron sin ejecutar, y puede haber caminos internos que nunca probaste.

Para este, se utilizan dos métodos:

1. **BASADOS EN ESPECIFICACIONES**
   Algunos métodos son:
   - **Partición de Equivalencias**
   - **Análisis de valores límites**
2. **BASADO EN LA EXPERIENCIA**
   Tiene que ver con el testing que hace el tester experimentado y sabe exactamente dónde ir para encontrar los defectos. Se basa en la experiencia.
   Algunos son:
   - Adivinanza de Defectos
   - Testing Exploratorio

Caja Blanca

Si se mira el código. El tester conoce la estructura interna y diseña los casos para recorrer esa estructura.

- Los **casos se derivan del código fuente** (y del diseño), no de los requerimientos.
- Busca **defectos internos**: ramas que no hacen nada, bucles mal armados, código muerto (que nunca se ejecuta), código malicioso, condiciones mal escritas.

- Se usa sobre todo en **pruebas unitarias y de integración**, porque ahí el que prueba (normalmente el desarrollador) tiene acceso al código.
- **Limitación**: si el código no implementa un requerimiento, la caja blanca no lo detecta (ejecutas todo lo que existe, pero lo que falta no aparece). Por eso se complementa con caja negra.
- Es **estático**: no ejecuta el código, sino que lo analiza leyéndolo (revisiones, inspecciones, recorridos, análisis estático con herramientas) → aunque sus métodos son dinámicos

Usa métodos de cobertura, los cuales son:

- **COBERTURA DE ENUNCIADOS O CAMINOS BÁSICOS**
- **COBERTURA DE SENTENCIAS**
- **COBERTURA DE DECISIÓN**
- **COBERTURA DE CONDICIÓN**
- **COBERTURA MÚLTIPLE**

CICLO DE TEST

Una versión del sistema no se prueba una sola vez. Cada vez que el equipo de desarrollo entrega una versión (un build) a testing, arranca un **ciclo de test**: se prueba, se encuentran defectos, se reportan, desarrollo los corrige y entrega una nueva versión, y vuelve a empezar.

Básicamente, un ciclo de pruebas abarca la ejecución de la totalidad de los casos de prueba establecidos aplicados a una misma versión del sistema a probar.

Existen estrategias que nos permiten decidir en cada ciclo que debemos de probar, que son las pruebas de regresión. Cuando se corrige un defecto, el cambio puede romper algo que antes funcionaba. Una prueba de regresión vuelve a ejecutar pruebas ya hechas para comprobar que los cambios no introdujeron defectos nuevos en partes que estaban bien.

1. **Pruebas CON REGRESIÓN**

Se prueba el defecto corregido, y se vuelven a probar otras funcionalidades.

Esto nos permite detectar si la corrección rompió otra cosa (ya que cuando solucionamos un defecto normalmente se introducen 2/3 defectos más) → es importante volver a verificar la nueva versión en su totalidad para asegurarnos de no haber introducido nuevos defectos.

Este tipo de pruebas es costoso y toma mucho tiempo por ciclo

2. **Pruebas SIN REGRESIÓN**

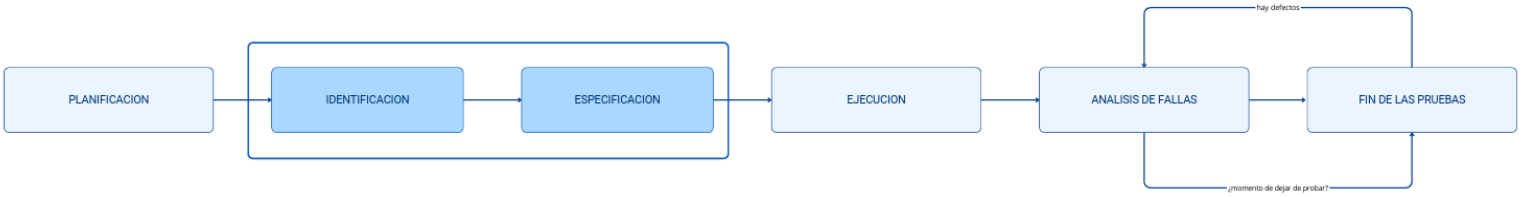
Se prueba solamente que el defecto corregido ya no ocurra. Si el defecto se arregla, entonces pasa y no vuelve a probarse en el próximo ciclo.

Al no probar el resto, la corrección puede introducir defectos nuevos sin que nadie se entere

La automatización ayuda a esta parte, dado que si hay CP automatizados, la regresión se ejecuta rápido y a bajo costo.

También puede usarse de forma mixta las dos estrategias, cada cantidad de ciclos se hace una prueba con regresión.

PROCESO DE PRUEBAS



En el proceso, se sigue:

- **PLANIFICACIÓN**: Parte más estratégica del testing, se genera un Plan de Prueba que define qué, cómo y cuándo se va a probar: alcance, recursos, ambiente, cronograma, criterios para terminar (que métodos a usar, como, entre otros)
    - Artefacto: **plan de pruebas** (documento que organiza el testing)
- **IDENTIFICACIÓN Y ESPECIFICACIÓN**: Se definen y describen los Casos de Prueba (diseño de CP, se preparan los datos, la base de datos, entre otros)
  En **identificación**, se analizan los requerimientos para identificar qué hay que probar, y se diseñan los casos de prueba (aplicando caja negra y/o blanca).
  En **especificación**, se detallan los casos (objetivo, datos de entrada, pasos, resultado esperado), se preparan los datos de prueba y la base de datos y se arma el ambiente.
    - Artefacto: **casos de prueba** (conjunto de CP diseñados)
- **EJECUCIÓN**: Ejecución de las pruebas, que se corren y se compara el resultado obtenido con el esperado (de manera manual o automatizada, o una combinación de ambas)
- **ANÁLISIS DE FALLAS**: Analiza si la solución/resultados de la ejecución de los casos de prueba fueron o no los esperados. Cuando un resultado no coincide, se analiza si es un defecto real o un problema del caso o del ambiente. Si es defecto, se lo reproduce y se reporta.
    - Artefacto: **Reporte de defectos** (registro de cada defecto encontrado)

  Si obtuvimos los resultados esperados, ocurre:
- **FIN DE LAS PRUEBAS**: Se evalúa si se cumplió el criterio de salida, se informan los resultados y se cierra el ciclo. Si no cumplió con lo esperado voy a hacer un análisis de fallas para corregirlo.
    - Artefacto: **informe de pruebas** (resultados finales y el estado de calidad)

Este ciclo entre ejecución y análisis de fallas (que forman el ciclo de prueba) se ejecuta tantas veces como sea necesario para llegar al criterio de aceptación que va a determinar luego el FIN DE LAS PRUEBAS.

¿Cuándo se determina el fin de las pruebas? Al no poder hacerse un testing exhaustivo, se definen criterios de salida o de finalización en el plan de pruebas.
Estas son, como por ejemplo:

- Se ejecutó cierto porcentaje de los casos planificados.
- Se alcanzó una cobertura mínima (de requerimientos o de código).
- No quedan defectos abiertos de severidad bloqueante o crítica, y los pendientes están por debajo de un límite acordado.
- La tasa de descubrimiento de defectos bajó (cada vez se encuentran menos).
- Se agotó el tiempo o el presupuesto.

¿Cuál es la diferencia entre verificación y validación?

Cuando se **verifica** se pregunta: “¿estamos construyendo el producto **correctamente**? ¿El sistema funciona bien respecto a su especificación?”. La verificación apunta a ver si estamos construyendo el sistema correctamente. Libre de defectos, apuntando a la perfección de su funcionalidad.

Cuando se **valida** se pregunta: “¿estamos construyendo el **producto correcto**? ¿El sistema es lo que el usuario necesita?”. La validación apunta a ver si estamos construyendo el sistema correcto, es decir, el sistema que cumple efectivamente con los requerimientos del cliente.